

Rappel sur les métriques SonarQube :

Métrique SonarQube	Définition simple	Ce que SonarQube détecte	Impact sur le code	Comment mesurer la qualité ?
Bug	Erreur pouvant provoquer un mauvais fonctionnement	Accès à undefined, logique incorrecte, dépendances manquantes	Risque de crash ou résultat incorrect	◇ Nombre de bugs ◇ Sévérité (Minor, Major, Critical) ◇ Reliability Rating (A → E)
Code Smell	Mauvaise pratique ou design perfectible	Code dupliqué, variable inutile, composant trop long	Code difficile à maintenir	◇ Nombre de code smells ◇ Technical Debt ◇ Maintainability Rating
Vulnerability	Faible de sécurité exploitable	XSS, injection, stockage non sécurisé	Risque de piratage	◇ Nombre de vulnérabilités ◇ Sévérité ◇ Security Rating
Security Hotspot	Zone sensible nécessitant revue humaine	Données utilisateur, HTML injecté, auth	Peut devenir une vulnérabilité	◇ Nombre de hotspots ◇ Statut (Reviewed / To Review)
Coverage	Pourcentage de code testé	Fonctions, branches, lignes non testées	Bugs non détectés	◇ % de lignes couvertes ◇ % de branches testées
Duplications	Code copié-collé	Blocs identiques ou similaires	Maintenance difficile	◇ % de duplication ◇ Nombre de blocs dupliqués
Complexity	Difficulté à comprendre le code	Trop de if, else, switch, boucles	Code difficile à lire	◇ Complexité cognitive ◇ Seuils par fonction
Maintainability	Facilité de maintenance	Basée sur code smells	Dettes techniques	◇ Maintainability Rating (A → E)
Reliability	Fiabilité du code	Basée sur bugs	Stabilité de l'app	◇ Reliability Rating (A → E)
Security	Niveau de sécurité global	Vulnérabilités + hotspots	Risque sécurité	◇ Security Rating (A → E)

Exercice 1 :

<pre>function Counter() { const [count, setCount] = useState(); return (<div> <p>{count.toFixed(2)}</p> <button onClick={() => setCount(count + 1)}></button> </div>); }</pre>	• Type : Bug • Métrique : Reliability • Pourquoi ? count est initialisé à undefined. Appeler toFixed() sur undefined provoque une erreur à l'exécution.
---	---

}	
---	--

Exercice 2 :

<div dangerouslySetInnerHTML={{ __html: userInput }} />	• Type : Vulnerability • Métrique : Security • Pourquoi ? Injection directe de HTML provenant de l'utilisateur → risque XSS.
---	--

Exercice 3 :

<pre>if (a) { if (b) { if (c) { if (d) { doSomething(); } } } }</pre>	• Type : Code Smell • Métrique : Complexity (Complexité cognitive) • Pourquoi ? Trop de conditions imbriquées → code difficile à comprendre et maintenir.
---	--

Exercice 4

<pre>function add(a, b) { return a + b; }</pre> Aucun test unitaire	• Type : Coverage • Métrique : Test Coverage • Pourquoi ? La fonction n'est pas couverte par des tests, donc baisse du coverage global.
--	---

Exercice 5

<pre>function ComponentA() { return <p>Hello World</p>; } function ComponentB() { return <p>Hello World</p>; }</pre>	• Type : Code Smell • Métrique : Duplications • Pourquoi ? Code identique copié-collé → difficile à maintenir.
---	---

Exercice 6 :

<pre>function Login() { const password = "123456"; return <input value={password} />; }</pre>	• Type : Vulnerability • Métrique : Security • Pourquoi ? Mot de passe codé en dur → faille de sécurité.
---	--

Exercice 7 :

<pre>useEffect(() => { if (a) { if (b) { if (c) { doSomething(); } } } }, [a, b, c]);</pre>	• Type : Code Smell • Métrique : Complexity • Pourquoi ? Logique trop imbriquée dans un hook → mauvaise lisibilité.
--	--

Exercice 8 :

Vous travaillez sur une application **React** nommée **react-shop-app**, développée en **JavaScript (JSX)**. L'objectif est de configurer **SonarQube** afin d'analyser la qualité du code (bugs, code smells, sécurité et coverage).

Détails du projet

- **Project Key** : react-shop-app
- **Project Name** : React Shop App
- **Version** : 1.0.0

Structure importante

- Code source : src/
- Tests unitaires : dans src/
- Fichiers de test : *.test.js, *.spec.js
- Rapport de couverture : coverage/lcov.info

Exclusions (ne pas analyser)

- node_modules/
- coverage/
- build/
- public/
- setupTests.js

Environnement SonarQube

- URL : http://localhost:9000
- Authentification : par **token**

Créer le fichier sonar-project.properties